U24AI047 AI LAB PRACTICAL-12

1. A tech hub has 6 different project teams (P1 to P6) that need to meet tomorrow. However, there are only 3 meeting rooms available (R1,R2,R3). Some teams have overlapping members and therefore cannot meet at the same time.

Constraints (Incompatibility List):

- P1 conflicts with: P2,P3,P6
- P2 conflicts with: P1,P3,P4
- P3 conflicts with: P1,P2,P5
- P4 conflicts with: P2,P6
- P5 conflicts with: P3,P6
- P6 conflicts with: P1,P4,P5

Task Requirements:

Initial State: Set the domain for every project variable to {R1,R2,R3}.

1. Constraint Enforcement: Apply AC-3 to this constraint graph.

2. Step-by-Step Trace: Provide a trace of the first 5 arc-reductions (e.g., "Arc (P1,P2) checked, no change").

3. Consistency Check: After running AC-3, determine if the problem is arc-consistent.

If we preemptively assign Team 1 to Room 1 , does AC-3 detect a failure, or does it leave a valid set of domains for the remaining rooms?

```python
# Teams and rooms
tList = ["P1", "P2", "P3", "P4", "P5", "P6"]
rooms = ["R1", "R2", "R3"]

# Conflicts
conf = {
    "P1": ["P2", "P3", "P6"],
    "P2": ["P1", "P3", "P4"],
    "P3": ["P1", "P2", "P5"],
    "P4": ["P2", "P6"],
    "P5": ["P3", "P6"],
    "P6": ["P1", "P4", "P5"]
}

# Generate arcs
arcs = []
for t1 in tList:
    for t2 in conf[t1]:
        arcs.append((t1, t2))

# Run AC-3 logic
def revised(x, y, d):
    rem = False
    new_d = []
    for v1 in d[x]:
        exists = False
        for v2 in d[y]:
            if v1 != v2:
                exists = True
                break
        if exists:
            new_d.append(v1)
        else:
            rem = True
    d[x] = new_d
    return rem

def run_ac3(d, a_list):
    q = []
    for item in a_list:
        q.append(item)
```

```python
    step_count = 1
    while len(q) > 0:
        x, y = q.pop(0)

        # Save the old domain before checking, so we can show the before/after
        old_domain = list(d[x])

        if revised(x, y, d):
            # --- THIS IS THE NEW TRACE SECTION ---
            print(f"\n[Check #{step_count}] REDUCTION TRIGGERED on Arc ({x}, {y})")
            print(f"  -> Because of {y}'s limitations, {x} loses a room!")
            print(f"  -> {x}'s domain changed: {old_domain} ---> {d[x]}")

            if len(d[x]) == 0:
                print(f"  -> FATAL ERROR: {x} has no valid rooms left. Halting.")
                return False

            print(f"  -> Cascade Effect: Re-checking {x}'s neighbors...")
            for nbr in conf[x]:
                if nbr != y:
                    q.append((nbr, x))
                    print(f"     Added Arc ({nbr}, {x}) back to the queue.")
            print("-" * 50)
            # -------------------------------------

        step_count += 1

    return True


# --- Running the Preemptive Assignment Case ---
print("\n--- Assigning P1 to R1 ---")
doms2 = {}
for t in tList:
    doms2[t] = ["R1", "R2", "R3"]
doms2["P1"] = ["R1"] # Force P1 into R1

print("Starting AC-3 Algorithm...\n")
if run_ac3(doms2, arcs):
    print("\nAC-3 finished successfully. Valid domains remain.")
    print("Final Domains:")
    for t in tList:
```

```
        print(f"  {t} : {doms2[t]}")
else:
    print("\nAC-3 detected failure!")
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-12\q1_better.py"
--- Assigning P1 to R1 ---
Starting AC-3 Algorithm...


[Check #4] REDUCTION TRIGGERED on Arc (P2, P1)
  -> Because of P1's limitations, P2 loses a room!
  -> P2's domain changed: ['R1', 'R2', 'R3'] ---> ['R2', 'R3']
  -> Cascade Effect: Re-checking P2's neighbors...
     Added Arc (P3, P2) back to the queue.
     Added Arc (P4, P2) back to the queue.
--------------------------------------------------

[Check #7] REDUCTION TRIGGERED on Arc (P3, P1)
  -> Because of P1's limitations, P3 loses a room!
  -> P3's domain changed: ['R1', 'R2', 'R3'] ---> ['R2', 'R3']
  -> Cascade Effect: Re-checking P3's neighbors...
     Added Arc (P2, P3) back to the queue.
     Added Arc (P5, P3) back to the queue.
--------------------------------------------------

[Check #14] REDUCTION TRIGGERED on Arc (P6, P1)
  -> Because of P1's limitations, P6 loses a room!
  -> P6's domain changed: ['R1', 'R2', 'R3'] ---> ['R2', 'R3']
  -> Cascade Effect: Re-checking P6's neighbors...
     Added Arc (P4, P6) back to the queue.
     Added Arc (P5, P6) back to the queue.
--------------------------------------------------

AC-3 finished successfully. Valid domains remain.
Final Domains:
  P1 : ['R1']
  P2 : ['R2', 'R3']
  P3 : ['R2', 'R3']
  P4 : ['R1', 'R2', 'R3']
  P5 : ['R1', 'R2', 'R3']
  P6 : ['R2', 'R3']
```

2. You are provided with a Sudoku puzzle classified as "Hard." Standard arc consistency often fails to solve such puzzles entirely but can significantly "thin out" the possibilities. Your task is to implement the AC-3 algorithm to see how much of the search space can be pruned before a human (or a backtracking algorithm) would need to start guessing.

Input Data: The following 81-character string representing the grid (0 denotes an empty cell):

0 0 0 0 0 6 0 0 0

0 5 9 0 0 0 0 0 8

2 0 0 0 0 8 0 0 0

0 4 5 0 0 0 0 0 0

0 0 3 0 0 0 0 0 0

0 0 6 0 0 3 0 5 0

0 0 0 0 0 7 0 0 0

0 0 0 0 0 0 0 0 0

0 0 0 0 5 0 0 0 2

Task Requirements:

1. Arc Generation: Automatically generate all 810+ binary constraints (arcs) based on Sudoku rules.

2. State Tracking: Output the number of values removed from all domains combined after the AC-3 algorithm terminates.

3. Visualization: Print the grid showing the remaining domain size for each cell (e.g., a "1" if solved, or "3" if three possibilities remain).

Key Question: Does AC-3 reduce any domain to zero (proving the puzzle unsolvable) or reduce all domains to one (solving it completely)?

```
# Sudoku Hard Puzzle AC-3

grid_rows = [
    "000006000",
    "059000008",
    "200008000",
```

```
    "045000000",
    "003000000",
    "006003050",
    "000007000",
    "000000000",
    "000050002"
]
s_grid = "".join(grid_rows)

# vars: 0 to 80
# doms
d = {}
for i in range(81):
    val = int(s_grid[i])
    if val == 0:
        d[i] = [1, 2, 3, 4, 5, 6, 7, 8, 9]
    else:
        d[i] = [val]

# Neighbors
def get_nbrs(i):
    res = set()
    r = i // 9
    c = i % 9

    # Row
    for col in range(9):
        if col != c:
            res.add(r * 9 + col)
    # Col
    for row in range(9):
        if row != r:
            res.add(row * 9 + c)
    # Box
    br = (r // 3) * 3
    bc = (c // 3) * 3
    for row in range(br, br + 3):
        for col in range(bc, bc + 3):
            idx = row * 9 + col
            if idx != i:
                res.add(idx)
    return res
```

```python
# Arcs
arcs = []
for i in range(81):
    nbrs = get_nbrs(i)
    for n in nbrs:
        arcs.append((i, n))

def revised(xi, xj):
    rem = False
    new_di = []
    for v1 in d[xi]:
        # is there a v2 in d[xj] such that v1 != v2
        ok = False
        for v2 in d[xj]:
            if v1 != v2:
                ok = True
                break
        if ok:
            new_di.append(v1)
        else:
            rem = True
    d[xi] = new_di
    return rem

# AC-3
q = []
for a in arcs:
    q.append(a)

removed = 0
while len(q) > 0:
    xi, xj = q.pop(0)
    old_len = len(d[xi])
    if revised(xi, xj):
        removed += (old_len - len(d[xi]))
        if len(d[xi]) == 0:
            # Silenced the mid-loop print to keep the final
output clean
            break
        for xk in get_nbrs(xi):
            if xk != xj:
```

```
                q.append((xk, xi))


# ==========================================
# MODIFIED OUTPUT SECTION
# ==========================================

print("\n" + "="*45)
print("AC-3 ALGORITHM RESULTS")
print("="*45)

# Task 1 Output
print("\nTask 1: Arc Generation")
print(f"- Successfully generated {len(arcs)} directed arcs")
print(f"  (This represents exactly {len(arcs)//2} binary
constraints).")

# Task 2 Output
print("\nTask 2: State Tracking")
print(f"- Total values removed from all domains: {removed}")

# Task 3 Output
print("\nTask 3: Visualization (Remaining Domain Sizes)")
for r in range(9):
    if r % 3 == 0 and r != 0:
        print("- - - + - - - + - - -")
    row_str = ""
    for c in range(9):
        if c % 3 == 0 and c != 0:
            row_str += "| "
        row_str += str(len(d[r*9 + c])) + " "
    print(row_str.strip())

# Key Question Output
print("\n" + "="*45)
print("ANSWER TO KEY QUESTION:")

if any(len(d[i]) == 0 for i in range(81)):
    print("- Outcome: AC-3 detected a failure (a domain
reached zero).")
    print("- Conclusion: The puzzle is mathematically
unsolvable.")
```

```python
elif all(len(d[i]) == 1 for i in range(81)):
    print("- Outcome: AC-3 solved the puzzle completely (all domains = 1).")
    print("- Conclusion: No human guessing or backtracking is required.")
else:
    print("- Outcome: AC-3 reduced the search space but did NOT solve it.")
    print("- Conclusion: No domain reached zero, but some remain > 1. A human or backtracking algorithm is needed to finish.")
print("="*45 + "\n")
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-12\tempCodeRunnerFile.py"
=============================================
AC-3 ALGORITHM RESULTS
=============================================

Task 1: Arc Generation
- Successfully generated 1620 directed arcs
  (This represents exactly 810 binary constraints).

Task 2: State Tracking
- Total values removed from all domains: 214

Task 3: Visualization (Remaining Domain Sizes)
5 4 4 | 7 6 1 | 7 6 6
5 1 1 | 5 5 3 | 6 6 1
1 4 3 | 6 5 1 | 7 6 7
- - - + - - - + - - -
4 1 1 | 6 6 3 | 7 7 5
4 5 1 | 8 7 5 | 7 7 5
4 5 1 | 6 6 1 | 6 1 4
- - - + - - - + - - -
7 6 4 | 7 7 1 | 7 6 6
8 7 5 | 7 7 4 | 8 7 7
7 6 4 | 6 1 3 | 7 7 1

=============================================
ANSWER TO KEY QUESTION:
- Outcome: AC-3 reduced the search space but did NOT solve it.
- Conclusion: No domain reached zero, but some remain > 1. A human or backtracking algorithm is needed to finish.
=============================================
```